

ARKANA



# Odoo V18 Technical Training

Prasetiya Mulya University

arkana.co.id



ODOO V19 • TECHNICAL TRAINING

# Day 1

## Foundations of Odoo Development

How Odoo Works + Build Your First Module

Prasetiya Mulya University  
arkana.co.id



# Odoo Technical Trainer



**Ardian Pramana**

Lead Developer @Arkana

# Rundown **Training** **Arkana**



- 09.00 - 10.30** : Sesi I
- 10.30 - 10.50** : Coffee Break
- 10.50 - 12.00** : Sesi II
- 12.00 - 13.15** : ISHOMA
- 13.15 - 15.20** : Sesi III
- 15.20 - 16.00** : Break Ashr
- 16.00 - 17.00** : Sesi IV

# ARKANA

## Follow our **Social Media**



@arkana\_academy



@arkana\_academy



Arkana Solusi Digital



@arkanasolusidigital

[www.arkana.co.id](http://www.arkana.co.id)

# Pretest Technical Training

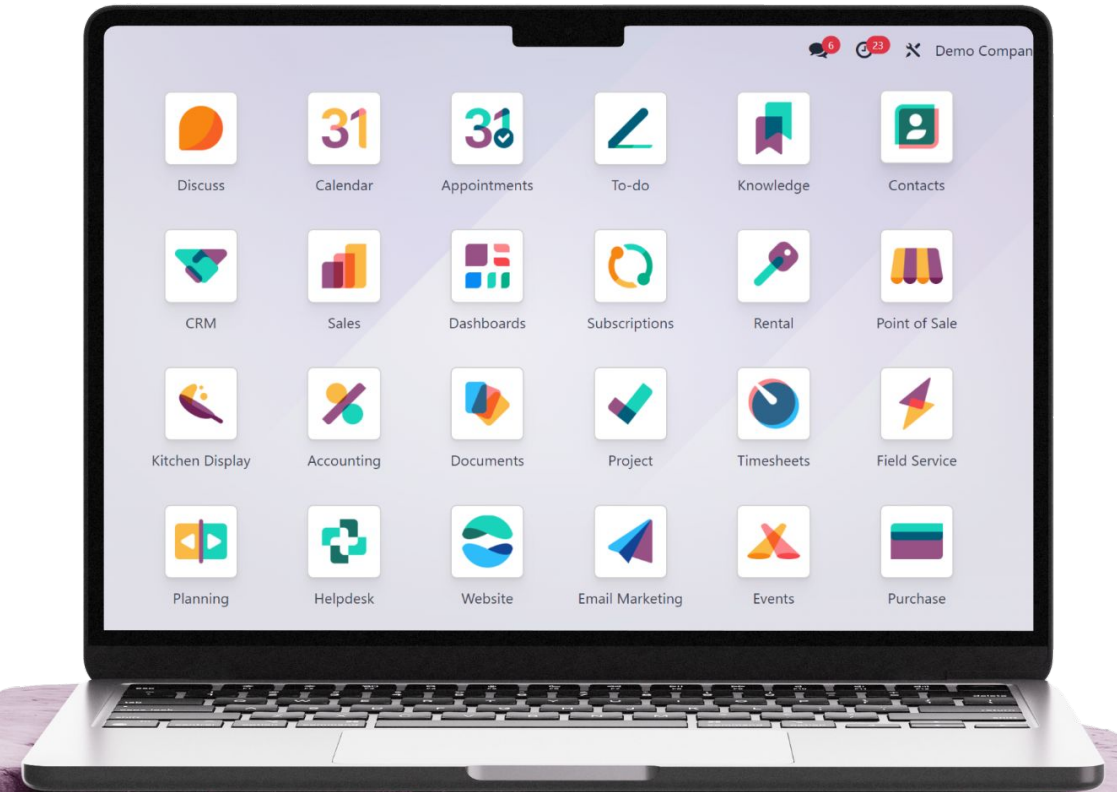
<https://forms.gle/aPFh9dDXKuJ9xc257>





# *Apa itu Odoo?*

Odoo itu platform bisnis all-in-one. Jadi bayangkan satu sistem yang bisa handle Sales, Inventory, HR, Accounting, CRM – semuanya terintegrasi dalam satu platform. Dulu namanya OpenERP, tapi sekarang sudah rebranding jadi Odoo karena scopenya sudah jauh lebih luas dari sekadar ERP.



# *Ada beberapa hal yang bikin Odoo menarik*

1. **Modular.** Artinya kita tidak harus install semuanya. Butuh Sales doang? Install modul Sales aja. Nanti butuh HR? Tinggal tambah. Ini fleksibel banget.
2. **Open source.** Kode sumbernya terbuka, kita bisa lihat, modifikasi, dan extend sesuai kebutuhan. Ini yang bikin Odoo populer di kalangan developer.
3. **Web-based.** Semua diakses lewat browser. Tidak perlu install aplikasi desktop apa pun.
4. **Pakai ORM (Object-Relational Mapping).** Jadi kita bisa interact dengan database tanpa nulis SQL langsung. Cukup pakai Python.
5. **Extensible.** Mudah banget diperluas dengan modul kustom. Inilah yang akan kita pelajari hari ini.



## TUJUAN &amp; SCOPE

# Apa yang Anda kuasai hari ini.



**Odoo sebagai application platform** — ERP = module di atas platform metadata-driven.



**Arsitektur Odoo** — UI · Python/ORM · PostgreSQL.



**Baca source standard module** — product.template · sale.order · res.partner.



**Buat custom module dari nol** — model · fields · data files · actions · menus.



**Metadata Odoo = data di PostgreSQL** — view, menu, action, security tersimpan sebagai rows.



**Fondasi Academy Management System** — titik awal aplikasi yang tumbuh 5 hari.

# Learning Journey: satu aplikasi yang terus tumbuh

1

DAY 1

## Data ada

module, ORM, fields,  
data files, actions,  
menus

2

DAY 2

## Data terhubung

views, relational fields,  
inheritance, computed,  
onchange

3

DAY 3

## Data berperilaku

constraints, advanced  
views, security, wizards

4

DAY 4

## Data berbicara

PDF, Excel, report  
inheritance, API  
consumer, RPC

5

DAY 5

## Data layak produksi

API provider,  
deployment,  
backup/restore, final

## DAY 1 • GOAL

**Definisi “selesai”  
hari ini.**

*Di akhir hari, peserta memiliki module yang berfungsi penuh — bukan sekadar teori.*

- ✓ module academy\_management berhasil dibuat
- ✓ academy.course berhasil dibuat
- ✓ academy.student berhasil dibuat
- ✓ data awal berhasil di-load
- ✓ menu Academy muncul
- ✓ submenu Courses & Students bisa dibuka
- ✓ create / edit record dari UI

## PART 1

# How Odoo Works

Mental model sebelum baris kode pertama

## PART 1 • WHY

***Anda jago  
programming. Itu  
justru jebakannya.***

*Di Odoo kita lebih banyak declare daripada implement. Banyak plumbing sudah disediakan framework.*

## REFLEKS DEVELOPER

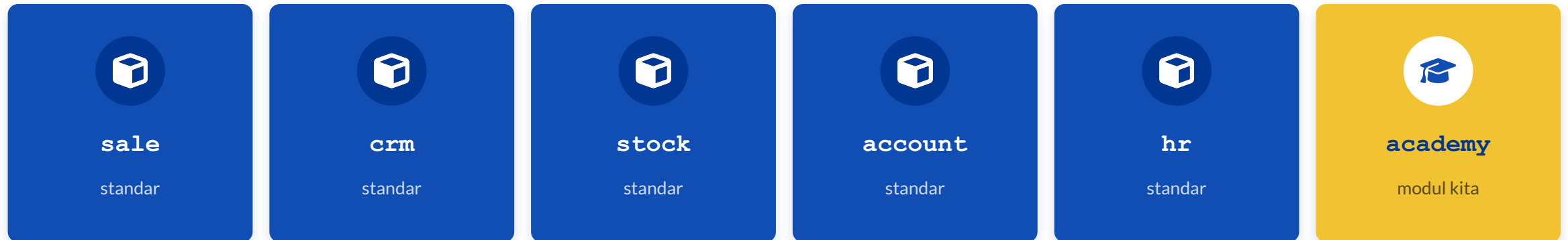
- ✗ Butuh tabel → migration manual
- ✗ Butuh API → controller + serializer
- ✗ Butuh UI → tulis HTML / JS
- ✗ Kustomisasi → fork & ubah core

## CARA ODOO

- ✓ Deklarasi model → ORM kelola schema
- ✓ Model + metadata → CRUD & RPC tersedia
- ✓ Deklarasi view (XML) → framework render
- ✓ Inherit & extend tanpa sentuh core

# Odoo bukan sekadar ERP — ia application platform

ERP = kumpulan module *di atas* platform metadata-driven. academy berdiri sejajar dengan module standar.



## ODOO FRAMEWORK

ORM • registry • environment • web / RPC layer • view engine • security • reporting

*Basis kita: Community (LGPL-3). Enterprise hanya menambah module di atas platform yang sama.*



# Satu request menembus lima layer



## Web Client (OWL)

JavaScript di browser — render UI, panggil server via RPC.

*black box • awareness only*



## Web / RPC Layer

HTTP routing (werkzeug), controllers, JSON-RPC / XML-RPC.



## Business Logic — Models

Python: fields, methods, constraints, workflow. Kode Anda.



## ORM Layer

recordset • env • registry. Generate SQL, enforce access.



## PostgreSQL

Tabel, foreign key, index — satu-satunya source of truth.

### ARAH ALIRAN

↓ **Request**  
turun dari UI ke DB

↑ **Data**  
naik sebagai recordset

# Baca source: product.template

addons/product/models/product\_template.py · disederhanakan

```
class ProductTemplate(models.Model):  
    _name = "product.template"  
    _description = "Product"  
  
    name = fields.Char(required=True)  
    list_price = fields.Float("Price")  
    type = fields.Selection(...)  
    categ_id = fields.Many2one(  
        "product.category")  
    taxes_id = fields.Many2many(...)
```



## Model standar = kode biasa

\_name, fields, \_description — sama seperti yang akan Anda tulis.



## Field types yang akan dipakai

Char, Float, Selection, Many2one, Many2many.



## Tidak ada API rahasia

Dasar membaca source Odoo dengan percaya diri.

**ERP MAPPING** academy.course meniru pola ini

# Coffee Break

**10.00 – 10.15**

Waktu Indonesia Barat (WIB)

## Tiga konsep inti: registry • env • recordset



### registry

Peta semua model hasil rakitan  
**SEMUA** module terinstall.

---

Bila 3 module meng-extend res.partner,  
registry menggabungkannya jadi satu  
class efektif saat startup.



### environment

Konteks eksekusi: cr (cursor), uid  
(user), context (dict).

---

Semua mengalir lewat self.env. Akses  
model lain: self.env['res.partner'].



### recordset

Unit kerja Odoo. self **SELALU**  
himpunan record.

---

Berisi 0, 1, atau N record. Tidak ada  
'satu objek = satu row'. Kode harus  
set-based.

## Baca source: sale.order — recordset & header-line

```
addons/sale/models/sale_order.py · disederhanakan
class SaleOrder(models.Model):
    order_line = fields.One2many(
        "sale.order.line", "order_id")
    amount_total = fields.Monetary(
        compute="_amount_all")

@api.depends("order_line.price")
def _amount_all(self):
    for order in self: # recordset
        order.amount_total = sum(
            order.order_line.mapped("price"))
```

### One2many = pola header-line

order\_line menghubungkan header ke baris. Exemplar termurni.

### for order in self

self adalah recordset. Aman untuk 1 maupun ribuan record.

### @api.depends

Computed field; recompute otomatis. .mapped() = operasi himpunan.

**ERP MAPPING** header-line ↔ academy.batch / enrollment

## Semuanya adalah module: kode + data + metadata

struktur folder

**academy\_management/**

- |— \_\_manifest\_\_.py
- |— \_\_init\_\_.py
- |— models/
- |— views/
- |— security/
- |— data/
- |— report/
- |— controllers/

```
__manifest__.py
{
    'name': 'Academy Management',
    'depends': ['base', 'mail'],
    'data': [
        'security/ir.model.access.csv',
        'views/academy_views.xml',
    ],
    'application': True,
}
```

**Inti:** manifest mengubah folder jadi module. File yang lupa didaftarkan di 'data' tidak akan dimuat. 'depends' menentukan load order.

## PART 1 • AHA MOMENT — METADATA-DRIVEN ARCHITECTURE

# Metadata adalah data.

View, menu, action, security bukan kode khusus — semuanya tersimpan sebagai records di PostgreSQL.



**View**

`ir.ui.view`



**Menu / Action**

`ir.actions.act_window`



**Security**

`ir.model.access`



**External ID**

`ir.model.data`

→ Konsekuensi: view bisa di-inherit tanpa menyentuh filenya, karena view itu sebuah record.

# Buktikan sendiri: semuanya hanya baris tabel

Query tabel `ir_*` – “magic” Odoo dalam bentuk rows. id 412 di `ir_ui_view` ditunjuk oleh `res_id` di `ir_model_data`.

## `ir_ui_view`

|                    |                                           |
|--------------------|-------------------------------------------|
| <code>id</code>    | <b>412</b>                                |
| <code>name</code>  | <code>sale.order.form</code>              |
| <code>model</code> | <code>sale.order</code>                   |
| <code>arch</code>  | <code>&lt;form&gt;...&lt;/form&gt;</code> |

Form yang Anda lihat = 1 baris di sini.

## `ir_actions_act_window`

|                        |                         |
|------------------------|-------------------------|
| <code>id</code>        | <b>388</b>              |
| <code>name</code>      | Quotations              |
| <code>res_model</code> | <code>sale.order</code> |
| <code>view_mode</code> | <code>list,form</code>  |

Menu “Quotations” memanggil action ini.

## `ir_model_access`

|                           |                               |
|---------------------------|-------------------------------|
| <code>name</code>         | <b>access_sale_user</b>       |
| <code>model_id</code>     | <code>sale.order</code>       |
| <code>group_id</code>     | <code>sales.group_user</code> |
| <code>perm r/w/c/u</code> | <code>1 · 1 · 1 · 0</code>    |

Hak akses CRUD = baris CSV → tabel.

## `ir_model_data`

|                     |                              |
|---------------------|------------------------------|
| <code>module</code> | <b>sale</b>                  |
| <code>name</code>   | <code>view_order_form</code> |
| <code>model</code>  | <code>ir.ui.view</code>      |
| <code>res_id</code> | 412 → kartu 1                |

External id = jembatan XML↔DB; kunci upgrade & uninstall.



# Yang terjadi di PostgreSQL: install • upgrade • uninstall



`install ( -i )`

- CREATE TABLE tiap model
- Muat semua file 'data'
- Catat ke ir\_model\_data



`upgrade ( -u )`

- Reload kode Python
- ORM diff & sesuaikan schema
- Re-load data (noupdate)



`uninstall`

- Lacak via ir\_model\_data
- DROP tabel & records
- Cascade ke data dependen



**Dev loop:** edit → `odoo -u academy_management` → test. Kenapa sering perlu -u, bukan sekadar restart? Karena banyak hal hidup sebagai DATA di DB — perubahan model/XML harus di-apply ulang lewat upgrade.

## PART 1 • ACADEMY CASE &amp; ERP MAPPING

***Kenali pattern,  
bukan hafal tabel.***

Analogi tiap model Academy dipetakan ke model standar.

| Academy Model                    | Analog Domain                                       | Pola Arsitektur                    | Modul      |
|----------------------------------|-----------------------------------------------------|------------------------------------|------------|
| <code>academy.course</code>      | <code>product.template</code>                       | Master / catalog data              | Sales      |
| <code>academy.student</code>     | <code>res.partner</code>                            | Partner universal                  | Contacts   |
| <code>academy.batch</code>       | <code>event.event</code>                            | Container: capacity, jadwal, state | Events     |
| <code>academy.enrollment</code>  | <code>event.registration</code>                     | Registrasi peserta                 | Events     |
| <code>academy.attendance</code>  | <code>stock.move</code> / <code>mail.message</code> | Transaksional volume tinggi        | Inventory  |
| <code>academy.certificate</code> | <code>account.move</code> + <code>QWeb</code>       | Dokumen bernomor + PDF             | Accounting |

**DOMAIN ANALOGY**

`batch`  $\approx$  `event.event` · `enrollment`  $\approx$  `event.registration`  
— container + registrasi.

**STRUCTURAL PATTERN**

Pola header–line termurni dari `sale.order` ↔ `sale.order.line`.

# ISHOMA

**11.50 – 13.05**

Waktu Indonesia Barat (WIB)

## PART 2

# Build Your First Module

Dari nol ke menu Academy yang berfungsi

## PART 2 • BUSINESS PROBLEM

## Di mana fitur baru hidup?

Academy butuh kelola Course & Student. Di mana kode ini ditaruh tanpa mengubah Odoo core?



**Jangan sentuh core** — Edit addons standar = mimpi buruk saat upgrade.



**Install / uninstall rapi** — Fitur bisa dipasang & dicabut tanpa sisa.



**Portable & ter-version** — Satu unit yang bisa di-Git, di-deploy, dibagikan.

*Jawaban Odoo: module — unit standar yang membungkus kode + data + metadata.*

## Scaffold sekali, lalu manifest yang menghidupkan

```
terminal
$ odoo-bin scaffold academy_management
    ./addons

academy_management/
    __manifest__.py
    __init__.py
    models/ views/ security/

$ odoo-bin -u academy_management --dev all
```

```
academy_management/__manifest__.py
{
    'name': 'Academy Management',
    'version': '18.0.1.0.0',
    'depends': ['base', 'mail'],
    'data': [
        'security/ir.model.access.csv',
        'data/academy_data.xml',
        'views/academy_views.xml',
        'views/academy_menus.xml',
    ],
    'application': True,
}
```

## PART 2 • LAB CHECKPOINT A

### Module hidup.

1. Scaffold module academy\_management ke folder addons.
2. Isi manifest: name, depends, application=True.
3. Update Apps List, install academy\_management (kosong).
4. Amati log startup 'loading academy\_management'.



#### SELESAI BILA

- academy muncul di Apps
- status Installed
- tidak ada error di log



10 menit • belum ada model — itu normal. Module kosong tetap valid.

## PART 2 • BUSINESS PROBLEM

**Course & student  
butuh struktur.**

Data harus ter-typed, tervalidasi, persistent, queryable. Tanpa menulis SQL DDL.

Reflex SQL:

```
CREATE TABLE academy_course (id serial, name varchar, price numeric, ...);
```

*Di Odoo Anda tidak menulis DDL. Deklarasikan model Python – ORM yang membuat & memelihara tabelnya.*



# Model academy.course

```
models/academy_course.py

class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"
    name = fields.Char(required=True)
    code = fields.Char(copy=False)
    description = fields.Html()
    duration_hours = fields.Float()
    price = fields.Monetary()
    currency_id = fields.Many2one(
        "res.currency")
    level = fields.Selection([
        ('beginner', 'Beginner'),
        ('advanced', 'Advanced')])
    active = fields.Boolean(default=True)
```

## currency\_id menemani Monetary

Tanpa currency\_id, Monetary tak bisa dirender. Pasangan wajib.

## copy=False pada code

Saat record diduplikasi, kode referensi tidak ikut tersalin.

## active = bisa di-archive

Mengikuti konvensi product.template.

# Model academy.student

```
models/academy_student.py

class AcademyStudent(models.Model):
    _name = "academy.student"
    _description = "Academy Student"
    name = fields.Char(required=True)
    email = fields.Char()
    phone = fields.Char()
    birthdate = fields.Date()
    gender = fields.Selection([
        ('male', 'Male'),
        ('female', 'Female')])
    active = fields.Boolean(default=True)
```



## KEPUTUSAN DESAIN

Untuk belajar fields, student dibuat sebagai model sendiri.

Di produksi, data orang hidup di res.partner (model partner universal). Relasinya dibahas Day 2.

# Common Attributes + Simple Fields

## COMMON ATTRIBUTES

`string · required · default · help · index · readonly · copy`

Contoh: `price = fields.Monetary(string="Price", help="Harga publik", required=True)`

## SIMPLE FIELDS

### Char / Text

string pendek / panjang

### Html

rich text

### Integer / Float

angka

### Monetary

uang (+ currency\_id)

### Boolean

true / false

### Date / Datetime

tanggal & waktu

### Selection

pilihan (key, label)

### —

Many2one → Day 2

# Reserved + Special Fields

## Reserved Fields (auto)

- `id`
- `create_uid` · `create_date`
- `write_uid` · `write_date`
- `display_name`

*Gratis dari ORM — jangan dideklarasikan ulang.*

## Special Fields

**`active`** — soft delete / archiving

**`state`** — workflow → Day 3

**`currency_id`** — pasangan Monetary

**`company_id`** — multi-company

*Konvensi semantik — dipakai seluruh Odoo.*

## PART 2 • LAB CHECKPOINT B

## Data ada di PostgreSQL.

1. Tulis `academy_course.py` & `academy_student.py`.
2. Import di `models/__init__.py` & `module __init__.py`.
3. Jalankan `odoo-bin -u academy_management`.
4. Di DBeaver, buka tabel `academy_course` & `academy_student`.
5. Cek kolom + auto fields (`id`, `create_date`, `write_uid`).

**SELESAI BILA**

- Tabel `academy_course` ada
- Tabel `academy_student` ada
- `level` (Selection) bertipe `varchar`
- Auto fields hadir

🕒 25 menit • *Selection* → *varchar*, *Monetary* → *numeric*. ORM yang menerjemahkan.

# Ashar Break

**15.10 – 15.45**

Waktu Indonesia Barat (WIB)

# Seed data yang ikut ter-install

Master data konsisten di tiap database & ter-version bersama module — lewat data files (XML/CSV).

```
data/academy_data.xml

<odoo>
  <record id="course_py_basics"
    model="academy.course">
    <field name="name">Python Basics</field>
    <field name="code">PY-101</field>
    <field name="level">beginner</field>
  </record>
</odoo>
```

## External id = id record

academy\_management.course\_py\_basics → ir\_model\_data (lihat Part 1).  
Jembatan XML ↔ DB.

## noupdate

noupdate="1" → data tak ditimpa saat -u.

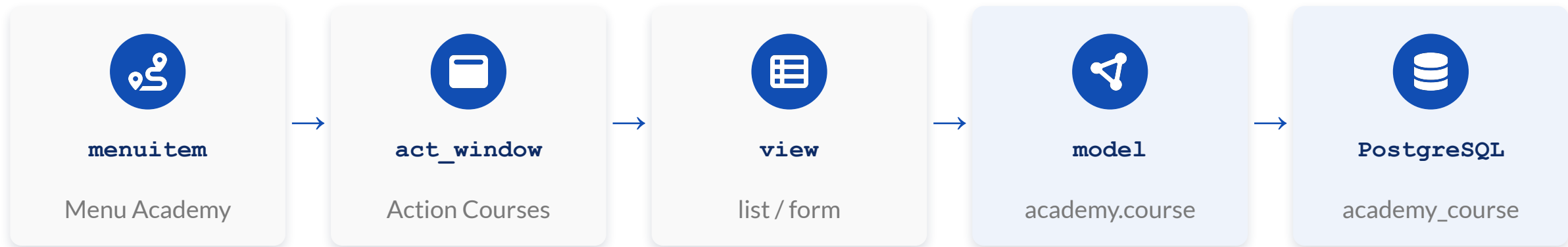
## data vs demo

'data' selalu dimuat; 'demo' hanya di database demo.

## Datanya ada — tapi tak terjangkau user

### BUSINESS PROBLEM

Record sudah di DB, tapi user belum punya cara mengakses. Odoo butuh rantai: menu → action → view untuk menampilkan model.





## Baca source: act\_window + menuitem

addons/sale/views/sale\_order\_views.xml · disederhanakan

```
<record id="action_orders"
    model="ir.actions.act_window">
    <field name="name">Quotations</field>
    <field name="res_model">sale.order</field>
    <field name="view_mode">list,form</field>
</record>

<menuitem id="menu_quotations"
    parent="menu_sale_root"
    action="action_orders" sequence="10"/>
```

### act\_window = jendela ke model

res\_model + view\_mode tentukan model & view.

### menuitem = tree knot

parent menyusun hierarki; action menautkan; sequence mengurutkan.

### v17+ : list, bukan tree

view\_mode pakai 'list'. Penting untuk Odoo 17/18/19.

## PART 2 • ACADEMY CASE — AHA MOMENT

# Tanpa satu baris view XML, menu Academy jalan.

```
views/academy_menus.xml
<record id="action_course"
  model="ir.actions.act_window">
  <field name="res_model">academy.course
    </field>
  <field name="view_mode">list,form</field>
</record>
<menuitem id="menu_root" name="Academy"/>
<menuitem id="menu_courses" name="Courses"
  parent="menu_root"
  action="action_course"/>
```

**View di-generate otomatis**

view\_mode='list,form' tanpa view XML → Odoo membuat list & form default dari fields.

**+ ir.model.access.csv**

Satu baris akses/model agar record terlihat.

**odoo-bin -u academy\_management**

Menu Academy muncul: Courses & Students bisa dibuka & diedit.

## Satu baris akses per model — wajib

Day 1 cukup practical minimum agar record tampil. Detail security (groups, record rules) → Day 3.

`security/ir.model.access.csv`

| id                     | name            | model_id:id           | group_id:id     | perm_read | perm_write | perm_create | perm_unlink |
|------------------------|-----------------|-----------------------|-----------------|-----------|------------|-------------|-------------|
| access_academy_course  | academy.course  | model_academy_course  | base.group_user | 1         | 1          | 1           | 1           |
| access_academy_student | academy.student | model_academy_student | base.group_user | 1         | 1          | 1           | 1           |



**Tanpa baris ini:** menu Academy tetap muncul, tapi user non-admin kena “Access Error” atau list kosong. Security bukan opsional.

*external id model\_<name> (mis. model\_academy\_course) dibuat otomatis dari nama model. base.group\_user = user internal.*

## PART 2 • LAB CHECKPOINT C

## Menu Academy berfungsi.

1. Buat ir.model.access.csv (course & student, base.group\_user).
2. Tulis act\_window untuk course & student.
3. Buat menu root Academy + Courses + Students.
4. Daftar semua file di manifest, lalu -u academy\_management.
5. Buka Academy → buat 1 course & 1 student dari UI.

**DELIVERABLE**

- Menu Academy tampil
- Courses: list & form jalan
- Students: list & form jalan
- Create/edit record dari UI

 30 menit • Inilah objective Day 1. Menu Academy jalan dengan Courses & Students = lulus.

## Jebakan tersering saat module pertama



### Lupa daftar file di manifest

Model/view/menu 'tidak muncul' padahal file ada.



### Lupa import di `__init__.py`

Model tidak terdaftar di registry.



### `ir.model.access` hilang

Access Error / list kosong untuk non-admin.



### Lupa `-u` setelah ubah kode

Perubahan tak terlihat. Restart saja tidak cukup.



### `view_mode 'tree'` di v17+

Gunakan 'list'. Tutorial lama menyesatkan.



### Monetary tanpa `currency_id`

Field uang gagal render.

*90% tiket “tidak muncul” berakhir di manifest / `__init__` / `ir.model.access`.*

# Data ada – dan bisa diakses



**Odoo = platform**

ERP = module di atas framework metadata-driven.



**ORM = registry • env • recordset**

self himpunan; kode set-based.



**Module = code + data + metadata**

di-install bersama; lifecycle lewat -u.



**Model → table via ORM**

fields = vocabulary; auto fields gratis.



**Data files = seed + external id**

tersimpan di ir\_model\_data.



**Menu → action → view → model**

+ ir.model.access; view auto-generated.

*Deliverable Day 1 tercapai: menu Academy dengan Courses & Students, bisa create/edit dari UI.*

## NEXT · DAY 2

**Data terhubung.**

Besok kita hubungkan model & percantik UX.



**Basic views** — form, list, search, kanban — ditulis tangan.



**Relational fields** — Many2one, One2many, Many2many.



**Model & View inheritance** — `_inherit` + XPath tanpa sentuh asli.



**Computed fields & defaults** — `@api.depends`, default values.



**Onchange** — interaksi form real-time.

WRAP-UP

# Review, QnA & Upcoming Preview

Pertanyaan, blocker lab, dan cuplikan Day 2



ARKANA



# Terima Kasih

Prasetiya Mulya University

[arkana.co.id](http://arkana.co.id)